

Regex und Parsing Übungen

ExtractLetters

Schreibe eine Methode, die alle Buchstaben aus einem String extrahiert und als neuen String zurückgibt.

Beispiel:

Eingabe: "?a b2c*d, e;"

Rückgabe: "abcde"

```
final class StringUtil {  
  
    static String extractLetters(String input) {  
        return null; // TODO  
    }  
}
```

ExtractLetters: Lösung

```
final class StringUtil {  
  
    static String extractLetters(String input) {  
        StringBuilder lettersOnly = new StringBuilder();  
  
        for (int i = 0; i < input.length(); i++) {  
            char c = input.charAt(i);  
            if (Character.isLetter(c)) {  
                lettersOnly.append(c);  
            }  
        }  
  
        return lettersOnly.toString();  
    }  
}
```

```
static String extractLetters(String input) { // alternative Lösung mit Regex  
    return input.replaceAll("[^a-zA-Z]", "");  
}
```

UrlParameterParser

Gegeben ist folgender **Unit Test** für die Klasse **UrlParameterParser**, der **aus einer URL alle Parameter als Map extrahiert**.
Implementiere die zugehörige Klasse **UrlParameterParser**!

Hinweise:

Parameter **starten nach** dem **ersten ?** in der URL und werden **anschließend** mit **&** **getrennt**.

Parameter haben immer einen **Key** und einen **Value**. Key und Value werden mit einem **=** **getrennt**.

Beispiel: `https://haw-hamburg.de?parameter1=value1¶meter2=value2¶meter3=value3`

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

import java.util.Map;

class UrlParameterParserTest {

    @Test
    void testUrlWithMultipleParameters() {
        // Arrange
        String url = "http://example.com/page?param1=value1&param2=value2";

        // Act
        Map<String, String> parameters = UrlParameterParser.parse(url);

        // Assert
        assertEquals(2, parameters.size());
        assertEquals("value1", parameters.get("param1"));
        assertEquals("value2", parameters.get("param2"));
        // Hinweis: Es können beliebig viele Parameter sein.
    }

    @Test
    void testUrlWithNoParameters() {
        // Arrange
        String url = "http://example.com/page";

        // Act
        Map<String, String> parameters = UrlParameterParser.parse(url);

        // Assert
        assertTrue(parameters.isEmpty());
    }
}
```

```
@Test
void testUrlWithSpecialCharacters() {
    // Arrange
    String url = "http://example.com/page?p1=special%20value&p2=another%20value";
    // Act
    Map<String, String> parameters = UrlParameterParser.parse(url);
    // Assert
    assertEquals(2, parameters.size());
    assertEquals("special%20value", parameters.get("p1"));
    assertEquals("another%20value", parameters.get("p2"));
}

@Test
void testUrlWithIncompleteParameters() {
    // Arrange
    String url = "http://example.com/page?something=value1&different=";
    // Act
    Map<String, String> parameters = UrlParameterParser.parse(url);
    // Assert
    assertEquals(2, parameters.size());
    assertEquals("value1", parameters.get("something"));
    assertEquals("", parameters.get("different"));
}

@Test
void testUrlWithInvalidFormat() {
    // Arrange
    String url = "http://example.com/page?param1";
    // Act
    Map<String, String> parameters = UrlParameterParser.parse(url);
    // Assert
    assertTrue(parameters.isEmpty());
}
}
```

UrlParameterParser : Lösung

```
class UrlParameterParser {  
  
    static Map<String, String> parse(String url) {  
        Map<String, String> parameters = new HashMap<>();  
  
        Pattern queryPattern = Pattern.compile("\\?(.*)$");  
        Matcher queryMatcher = queryPattern.matcher(url);  
  
        if (queryMatcher.find()) {  
            String query = queryMatcher.group(1);  
  
            Pattern paramPattern = Pattern.compile("([^&=]+)=([^&]*)");  
            Matcher paramMatcher = paramPattern.matcher(query);  
  
            while (paramMatcher.find()) {  
                String key = paramMatcher.group(1);  
                String value = paramMatcher.group(2);  
                parameters.put(key, value);  
            }  
        }  
  
        return parameters;  
    }  
}
```

Verwendungszweck

Der Semesterbeitrag der HAW Hamburg wird jedes Semester mit einem speziellen Verwendungszweck überwiesen:

Verwendungszweck für das Sommersemester 2024:

Semesterkennzahl 20241 und Ihre Matrikelnummer (bitte lassen Sie kein Leerzeichen dazwischen!) Name, Vorname (bitte lassen Sie ein Leerzeichen zwischen der Matrikelnummer und Ihrem Namen!)

Beispiel: 202412567890 Mustermann, Max

Implementiere einen **Parser**, der den **nachfolgenden Unit Test besteht** sowie das **nachfolgende Enum und Record** verwendet.

Verwendungszweck – Hilfsklassen

Verwende das hier aufgeführte Record und Enum.

```
record Verwendungszweck(  
    int year,  
    Semester semester,  
    String matriculationId,  
    String firstName,  
    String lastName) {}
```

```
enum Semester {  
  
    SOSE(1),  
    WISE(2);  
  
    private final int id;  
  
    Semester(int id) {  
        this.id = id;  
    }  
  
    static Semester of(int id) {  
        for (var semester : Semester.values()) {  
            if (semester.id == id) {  
                return semester;  
            }  
        }  
        throw new IllegalArgumentException("Ungültige Semester-ID: " + id);  
    }  
  
    @Override  
    public String toString() {  
        return "%s{id=%d}".formatted(this.name(), this.id);  
    }  
}
```

Verwendungszweck – Unit Test

```
class VerwendungszweckParserTest {

    private VerwendungszweckParser parser;

    @BeforeEach
    void setUp() {
        parser = new VerwendungszweckParser();
    }

    @Test
    void parseValidVerwendungszweckWithSoSe() {
        // Arrange
        String input = "202412345678 Musk, Elon";
        // Act
        Verwendungszweck actual = parser.parse(input);
        // Assert
        assertEquals(2024, actual.year());
        assertEquals(Semester.SOSE, actual.semester());
        assertEquals("2345678", actual.matriculationId());
        assertEquals("Elon", actual.firstName());
        assertEquals("Musk", actual.lastName());
    }

    @Test
    void parseValidVerwendungszweckWithWiSe() {
        // Arrange
        String input = "202328765432 Theoden, Eowyn";
        // Act
        Verwendungszweck actual = parser.parse(input);
        // Assert
        assertEquals(2023, actual.year());
        assertEquals(Semester.WISE, actual.semester());
        assertEquals("8765432", actual.matriculationId());
        assertEquals("Eowyn", actual.firstName());
        assertEquals("Theoden", actual.lastName());
    }

    @Test
    void parseValidVerwendungszweckWithSpecialName() {
        // Arrange
        String input = "202228765432 von Lauch, Lurch-Ludwig";
        // Act
        Verwendungszweck actual = parser.parse(input);
        // Assert
        assertEquals(2022, actual.year());
        assertEquals(Semester.WISE, actual.semester());
        assertEquals("8765432", actual.matriculationId());
        assertEquals("Lurch-Ludwig", actual.firstName());
        assertEquals("von Lauch", actual.lastName());
    }
}
```

```
@Test
void invalidVerwendungszweckIsNull() {
    assertThrows(IllegalArgumentException.class, () -> parser.parse(null));
}

@Test
void invalidEmptyVerwendungszweck() {
    // Arrange
    String emptyString = "";
    // Act and Assert
    assertThrows(IllegalArgumentException.class, () -> parser.parse(emptyString));
}

@Test
void invalidName() {
    // Arrange
    String invalidName = "202412345678 Breitner, Paul, Hans";
    // Act and Assert
    assertThrows(IllegalArgumentException.class, () -> parser.parse(invalidName));
}

@Test
void invalidNumberWithLetters() {
    // Arrange
    String numberWithLetters = "abcd12345678 Mustermann, Max";
    // Act and Assert
    assertThrows(IllegalArgumentException.class, () -> parser.parse(numberWithLetters));
}

@Test
void invalidNumberTooShort() {
    // Arrange
    String numberTooShort = "2024123456 Mustermann, Max";
    // Act and Assert
    assertThrows(IllegalArgumentException.class, () -> parser.parse(numberTooShort));
}

@Test
void invalidNumberTooLong() {
    // Arrange
    String numberTooLong = "2024123456789 Mustermann, Max";
    // Act and Assert
    assertThrows(IllegalArgumentException.class, () -> parser.parse(numberTooLong));
}

@Test
void invalidSemesterId() {
    // Arrange
    String invalidSemesterId = "202432345678 Mustermann, Max";
    // Act and Assert
    assertThrows(IllegalArgumentException.class, () -> parser.parse(invalidSemesterId));
}
```


Verwendungszweck: Lösung

```
class VerwendungszweckParser {  
  
    Verwendungszweck parse(String input) {  
        if (input == null || input.isEmpty()) {  
            throw new IllegalArgumentException("Input cannot be null or empty");  
        }  
  
        Matcher matcher = Pattern.compile("(\\d{4})(\\d)(\\d{7}) ([^,]+), ([^,]+)").matcher(input);  
        if (!matcher.matches()) {  
            throw new IllegalArgumentException("Invalid input format");  
        }  
  
        int year = Integer.parseInt(matcher.group(1));  
        int semesterId = Integer.parseInt(matcher.group(2));  
        Semester semester = Semester.of(semesterId);  
        String matriculationId = matcher.group(3);  
        String lastName = matcher.group(4);  
        String firstName = matcher.group(5);  
  
        return new Verwendungszweck(year, semester, matriculationId, firstName, lastName);  
    }  
}
```

Dollar->Euro Converter

Implementiere folgende Methode `convertDollarsToEuros`: Konvertiere eine gegebene Collection von Strings im Dollar-Währungsformat (z. B. "\$1,234.56") in ein Euro-Währungsformat (z. B. "1.234,56 €"). Beachte den Unterschied zwischen Punkt und Komma!

Jede ungültige Dollar-Währung wird ignoriert und nicht in die resultierende Collection aufgenommen. Verwende die Hilfsmethode `isValidDollar`, um zu prüfen, ob eine Dollar-Währung gültig ist.

Rückgabe: Eine neue Collection mit umgerechneten EU-Währungen.

Prüfe deine Lösung mit folgenden Unit Test.

```
class DollarToEuroConverter {  
    Collection<String> convertDollarsToEuros(Collection<String> dollars) {  
        return null; // TODO  
    }  
  
    private static boolean isValidDollar(String dollar) { // returns true if dollar has a valid format, false otherwise.  
        return dollar.matches("^\\$[\\d,]+\\.\\d{2}$");  
    }  
}
```

Dollar->Euro Converter: Unit Test

```
class DollarToEuroConverterTest {  
    private final DollarToEuroConverter converter = new DollarToEuroConverter();  
  
    @Test  
    void testConvertDollarsToEurosWithValidCurrencies() {  
        var dollars = List.of("$1,234.56", "$2,345.67");  
  
        var result = converter.convertDollarsToEuros(dollars);  
  
        assertEquals(List.of("1.234,56 €", "2.345,67 €"), result);  
    }  
  
    @Test  
    void testConvertDollarsToEurosWithEmptyInput() {  
        List<String> dollars = List.of();  
  
        var actual = converter.convertDollarsToEuros(dollars);  
  
        assertEquals(List.of(), actual);  
    }  
  
    @Test  
    void testConvertDollarsToEurosWithMixedValidityCurrencies() {  
        var dollars = List.of("$1,000.00", "invalid", "$2,500.50");  
  
        var actual = converter.convertDollarsToEuros(dollars);  
  
        assertEquals(List.of("1.000,00 €", "2.500,50 €"), actual);  
    }  
  
    @Test  
    void testConvertDollarsToEurosWithInvalidNumbers() {  
        var dollars = List.of(  
            "$2.a00,50",  
            "2,500.50",  
            "$2.500,50",  
            "$2,500,50",  
            "$2.500.50",  
            "$2.500.",  
            "$2.500.0");  
  
        var actual = converter.convertDollarsToEuros(dollars);  
  
        assertEquals(List.of(), actual);  
    }  
}
```

Dollar->Euro Converter: Lösung

```
class UsCurrencyConverter {  
  
    Collection<String> convertDollarsToEuros(Collection<String> dollars) {  
        var euros = new ArrayList<String>();  
  
        for (String usCurrency : dollars) {  
            try {  
                if (isValidUsCurrency(usCurrency)) {  
                    String euro = convertToEu(usCurrency);  
                    euros.add(euro);  
                }  
            } catch (Exception e) {  
                // Ignore  
            }  
        }  
  
        return euros;  
    }  
  
    private static boolean isValidUsCurrency(String dollar) {  
        return dollar.matches("^\\$[\\d,]+\\.\\d{2}\\$");  
    }  
  
    private static String convertToEu(String dollar) {  
        String noDollarSignWithPeriodAndCommaSwapped = dollar  
            .replace("$", "")  
            .replace(".", ",")  
            .replaceFirst(",", ".");  
        return noDollarSignWithPeriodAndCommaSwapped + " €";  
    }  
}
```

Klammerama

Implementiere eine Methode `isValid`, die eine Zeichenkette mit verschiedenen Klammerarten entgegennimmt und validiert, ob jede **geöffnete Klammer** korrekt mit einer **entsprechenden schließenden Klammer** in **Reihenfolge gepaart** ist. Das heißt, dass jede Klammer, die zuletzt geöffnet wurde, als erste wieder geschlossen werden muss.

Verwende **folgenden Unit Test**, um deine **Lösung** zu **überprüfen**.

Hinweis: Die Aufgabe lässt sich nicht mit Regex lösen, da Regex die Verschachtelung nicht nachverfolgen kann.

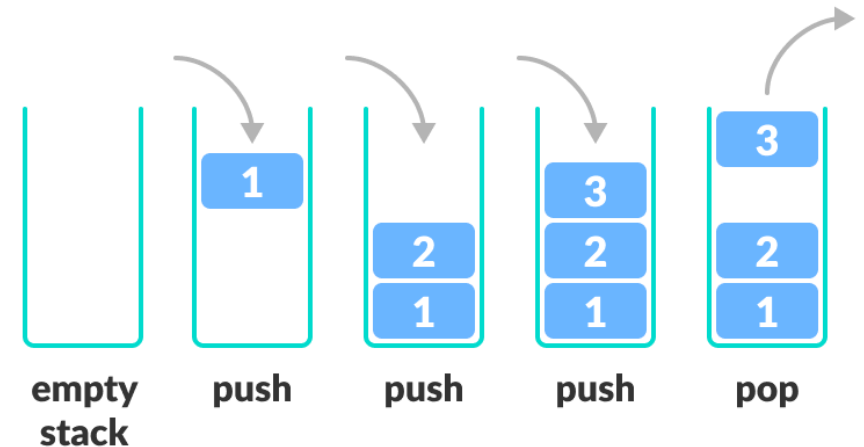
Tipp: Nutze ein [Deque](#) (bzw. Stack).

Beispiele:

```
validator.isValid("abc");           // true, da keine Klammern

validator.isValid("(");              // true
validator.isValid("{}");             // true
validator.isValid("a(b[c{d}e]f)g");  // true
validator.isValid("([[]])");         // true

validator.isValid("(");              // false, fehlende, schließende Klammer
validator.isValid("[]");             // false, fehlende öffnende Klammer
validator.isValid("a(b{c[d]e}f)");   // false
validator.isValid("(){}");           // false, wegen falscher Reihenfolge
validator.isValid("{}[]");           // false, fehlende öffnende Klammer
```



Klammerama – Unit Test

```
class BracketsValidatorTest {

    private BracketsValidator validator;

    @BeforeEach
    void setUp() {
        validator = new BracketsValidator();
    }

    @Test
    void noBrackets() {
        assertTrue(validator.isValid("abc"));
    }

    @Test
    void oneOpeningOneClosing() {
        assertTrue(validator.isValid("{}"));
    }

    @Test
    void twoOpeningTwoClosing() {
        assertTrue(validator.isValid("{}{}"));
    }

    @Test
    void threeOpeningThreeClosing() {
        assertTrue(validator.isValid("{}{}{}"));
    }

    @Test
    void threeOpeningThreeClosingSameBrackets() {
        assertTrue(validator.isValid("{}({})"));
    }

    @Test
    void threeOpeningThreeClosingSameBracketsWithCharacters() {
        assertTrue(validator.isValid("a(b(c(d)e)f)g"));
    }

    @Test
    void threeOpeningThreeClosingWithCharacters() {
        assertTrue(validator.isValid("a(b[c(d)e]f)g"));
    }

    @Test
    void fourOpeningFourClosing() {
        assertTrue(validator.isValid("{}({}{}))"));
    }

    @Test
    void fourOpeningFourClosingNotNested() {
        assertTrue(validator.isValid("{}{}{}{}"));
    }

    @Test
    void fourOpeningFourClosingNotNestedWithCharacters() {
        assertTrue(validator.isValid("a[b]c[d]e(f)g"));
    }
}
```

```
@Test
void threeOpeningThreeClosingNotNestedWithCharacters() {
    assertTrue(validator.isValid("a[b]c[d]e(f)g"));
}

@Test
void oneOpeningNoClosing() {
    assertFalse(validator.isValid("{}"));
}

@Test
void oneOpeningNoClosingWithCharacters() {
    assertFalse(validator.isValid("abc(def)"));
}

@Test
void noOpeningOneClosing() {
    assertFalse(validator.isValid("{}"));
}

@Test
void noOpeningOneClosingWithCharacters() {
    assertFalse(validator.isValid("abc[def]"));
}

@Test
void oneOpeningTwoClosingSameBrackets() {
    assertFalse(validator.isValid("{}{}"));
}

@Test
void oneOpeningTwoClosing() {
    assertFalse(validator.isValid("{}{}"));
}

@Test
void oneOpeningTwoClosingNonNested() {
    assertFalse(validator.isValid("{}{}"));
}

@Test
void oneOpeningTwoClosingWithCharacters() {
    assertFalse(validator.isValid("a[b]c[d]"));
}

@Test
void twoOpeningOneClosing() {
    assertFalse(validator.isValid("{}{}"));
}

@Test
void twoOpeningOneClosingSameBrackets() {
    assertFalse(validator.isValid("{}{}"));
}

@Test
void twoOpeningOneClosingNonNested() {
    assertFalse(validator.isValid("{}{}"));
}
```

```
@Test
void twoOpeningOneClosingWithCharacters() {
    assertFalse(validator.isValid("a[b[c]d]"));
}

@Test
void twoOpeningTwoClosingWrongOrder() {
    assertFalse(validator.isValid("{}{}"));
}

@Test
void twoOpeningTwoClosingWrongOrderWithCharacters() {
    assertFalse(validator.isValid("a(b[c]d)e"));
}

@Test
void threeOpeningTwoClosing() {
    assertFalse(validator.isValid("{}{}{}"));
}

@Test
void threeOpeningThreeClosingWrongOrder() {
    assertFalse(validator.isValid("{}{}{}"));
}

@Test
void threeOpeningThreeClosingWrongOrderWithCharacters() {
    assertFalse(validator.isValid("a(b[c[d]e]f)g"));
}

@Test
void threeOpeningFourClosing() {
    assertFalse(validator.isValid("{}{}{}{}"));
}

@Test
void threeOpeningFourClosingMixedNested() {
    assertFalse(validator.isValid("{}{}{}{}"));
}
}
```

Klammerama – Lösung

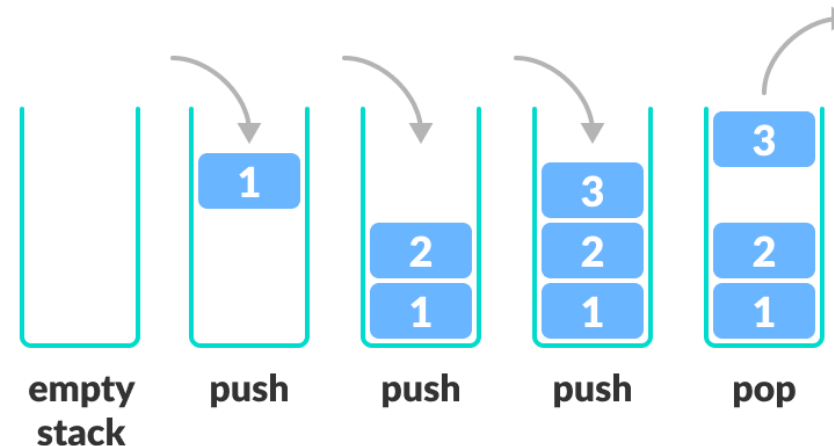
```
class BracketsValidator {  
  
    private static final Map<Character, Character> VALID_BRACKETS = Map.of(  
        '(', ')',  
        '{', '}',  
        '[', ']'  
    );  
  
    boolean isValid(String input) {  
        Deque<Character> stack = new LinkedList<>();  
        for (char character : input.toCharArray()) {  
            if (isOpeningBracket(character)) {  
                char openingBracket = character;  
                stack.push(openingBracket);  
            } else if (isClosingBracket(character)) {  
                if (noOpeningBracketExists(stack)) {  
                    return false;  
                }  
  
                char closingBracket = character;  
                char lastAddedOpeningBracket = stack.pop();  
                if (!matches(lastAddedOpeningBracket, closingBracket)) {  
                    return false;  
                }  
            }  
        }  
        return stack.isEmpty();  
    }  
  
    private boolean isOpeningBracket(char character) {  
        return VALID_BRACKETS.containsKey(character);  
    }  
  
    private boolean isClosingBracket(char character) {  
        return VALID_BRACKETS.containsValue(character);  
    }  
  
    private static boolean noOpeningBracketExists(Collection<Character> stack) {  
        return stack.isEmpty();  
    }  
  
    private boolean matches(char openingBracket, char actualClosingBracket) {  
        char expectedClosingBracket = VALID_BRACKETS.get(openingBracket);  
        return Objects.equals(expectedClosingBracket, actualClosingBracket);  
    }  
}
```

Hinweis:

LinkedList implementiert das Interface Deque.

Deque hat die Klasse Stack in Java abgelöst und verfügt über spezielle Methoden:

- push(E element) fügt ein Element an letzter Stelle hinzu.
- pop() entfernt das zuletzt hinzugefügte Element.
- peek() gibt das zuletzt hinzugefügte Element zurück (ohne es zu entfernen).



CommentBlockValidator

Implementiere eine Methode, die prüft, ob in einem String auf alle **öffnenden Kommentarblöcke** `/*` genauso viele **schließende Kommentarblöcke** `*/` folgen. Es gibt **keine Überlappungen** zwischen öffnenden und schließenden Kommentarzeichen:

- Der String `"/*/` gilt nur als öffnender Kommentarblock, aber nicht als schließender.
- Der String `"*/"` gilt nur als schließender Kommentarblock, aber nicht als öffnender.

Verwende **folgenden Unit Test**, um deine **Lösung** zu **überprüfen**.

Hinweis: Die Aufgabe lässt sich nicht mit Regex lösen, da Regex die Verschachtelung nicht nachverfolgen kann.

Beispiele:

```
var validator = new CommentBlockValidator();
```

```
validator.checkComments("/* Kommentar /* ungeschlossener Kommentarblock */"); // false  
validator.checkComments("Kommentar /*/* ungeschlossener */ Kommentarblock"); // false  
validator.checkComments("Ungeschlossener *//* Kommentarblock"); // false  
validator.checkComments("/*/"); // false
```

```
validator.checkComments("/* Kommentar 1 */ normaler Text /* Kommentar 2 */"); // true  
validator.checkComments("Dies ist /* ein /* Inception */ Kommentar */ !"); // true  
validator.checkComments("Dies ist kein Kommentar */, aber dies ist /* ein Kommentar */."); // true  
validator.checkComments("Dies /**/ ist ein Minikommentar."); // true  
validator.checkComments("Normaler Text ohne Kommentarblock"); // true
```

```
class CommentBlockValidator {
```

```
    boolean checkComments(String input) {
```

```
        return false; // TODO
```

```
    }
```

```
}
```


CommentBlockValidator: Unit Test

```
class CommentBlockValidatorTest {

    private CommentBlockValidator validator;

    @BeforeEach
    void setup() {
        validator = new CommentBlockValidator();
    }

    @Test
    void nomalText() {
        assertTrue(validator.checkComments("ohne Blub"));
    }

    @Test
    void multipleComments() {
        assertTrue(validator.checkComments("/ * Blub 1 * / Text / * Blub 2 * / and / * Blub 3 * /"));
    }

    @Test
    void inceptionComments() {
        assertTrue(validator.checkComments("Dies ist / * ein / * Inception * / Blub * / !"));
    }

    @Test
    void startWithClosingCommentsIsIgnored() {
        assertTrue(validator.checkComments("Dies * / ist / * ein Blub * / ."));
    }

    @Test
    void moreOpeningsThanClosings() {
        assertFalse(validator.checkComments("/ * Blub 1 / * ungeschlossener Blub * /"));
    }

    @Test
    void moreClosingsThanOpenings() {
        assertFalse(validator.checkComments("/ * Blub 1 ungeschlossener / * Blub * /"));
    }

    @Test
    void miniComment() {
        assertTrue(validator.checkComments("/ ** / blub"));
        assertTrue(validator.checkComments("/ *** / blub"));
    }

    @Test
    void starSlashSlashStar() {
        assertFalse(validator.checkComments("/ ** / * / blub"));
    }

    @Test
    void twoClosingCommentsSurroundedByStars() {
        assertTrue(validator.checkComments("/ *** / ** blub"));
    }

    @Test
    void starSlashStar() {
        assertTrue(validator.checkComments("/ ** / blub"));
        assertTrue(validator.checkComments("/ * / * / blub"));
    }

    @Test
    void slashStarSlash() {
        assertFalse(validator.checkComments("/ * / blub"));
        assertFalse(validator.checkComments("/ * / * / * / blub"));
    }
}
```

CommentBlockValidator: Lösung

```
class CommentBlockValidator {

    private static final char SLASH = '/';
    private static final char STAR = '*';

    private String input;

    boolean checkComments(String input) {
        this.input = input;
        int commentCount = 0;
        int index = 0;

        while (hasNextChar(index)) {
            if (isOpeningComment(index)) {
                commentCount++;
                index++;
            } else if (isClosingComment(index)) {
                commentCount = decrementIfOpeningExists(commentCount);
                index++;
            }
            index++;
        }

        return commentCount == 0;
    }

    private boolean hasNextChar(int index) {
        return (index + 1) < input.length();
    }

    private boolean isOpeningComment(int index) {
        return currentChar(index) == SLASH && nextChar(index) == STAR;
    }

    private boolean isClosingComment(int index) {
        return currentChar(index) == STAR && nextChar(index) == SLASH;
    }

    private char currentChar(int index) {
        return input.charAt(index);
    }

    private char nextChar(int index) {
        return input.charAt(index + 1);
    }

    private static int decrementIfOpeningExists(int commentCount) {
        if (commentCount > 0) {
            return commentCount - 1;
        }
        return commentCount;
    }
}
```